

Parallel Triangle Counting Algorithms

Michele Carloni AND Riccardo Revalor

(s338856@studenti.polito.it)

(s339423@studenti.polito.it)

Politecnico di Torino, Computer Engineering

Abstract

This paper presents a comprehensive analysis of various algorithms for triangle counting in graphs. We investigate the *Forward algorithm*, implemented with both *node-centric* and *edge-centric* iteration strategies. For each of these, we test two distinct data representations: an *adjacency list* and an *adjacency vector*. All Forward algorithm variants are evaluated across *sequential*, *parallel C++ (CPU threads)*, and *parallel CUDA (GPU threads)* implementations. Furthermore, we explore a *third algorithm* based on *matrix multiplication*, which is also implemented and benchmarked using both *parallel C++* and *parallel CUDA* approaches. This work provides insights into the performance characteristics and scalability of these diverse methodologies for triangle enumeration.

 [Click here for the GitHub Repository of the project](#)

0 Mathematical definition of the problem

Suppose that we have at our disposal a given graph $G = (V, E)$, where G is the graph itself, V denotes the set of all vertices (also called nodes), and E represents the set of edges (also called links or arcs). An edge connects two nodes between them. The primary goal is to detect and count the number of triangles that can be formed inside the graph. More specifically, a **triangle** in graph $G = (V, E)$ is a set of three distinct vertices $\{u, v, w\} \subseteq V$ such that the edges (u, v) , (v, w) , and (w, u) all exist in the set of edges E . This problem can be solved essentially using two distinct main approaches:

0.1 Graph Traversal/Iterative Methods

This category encompasses algorithms like the **Node Iterator**[1, 2] or the **Edge Iterator**[1, 2] that directly explore the graph, and at each iteration they try to look for the specific pattern that defines a triangle (three mutually connected vertices, as said before). They operate by iterating through parts of the graph (by scanning the list of either its nodes or edges) and checking for connections.

0.2 Linear Algebra/Matrix Methods

In this category, we mainly recall the **Matrix Multiplication** approach[2], which only works if the graph is represented by an adjacency matrix, which leverages matrix operations, specifically cubing (i.e. matrix multipli-

cation) and trace computation, to get the exact number of triangles which can be created. Given the graph adjacency matrix A , this number is computed as

$$\frac{1}{6} \text{Trace}(A^3)$$

In graph terms, this trace represents the total count of all closed walks of length 3 in the graph, summed across all possible starting/ending vertices.

1 Node Iterator Algorithm, Forward Algorithm

The simplest and most naive version of the Node Iterator Algorithm iterates over all nodes in the graph and, for each pair of neighbors of each node, checks whether they are connected by an edge and they share a common neighbor. For each vertex, its **degree** $d(v)$ denotes its neighbors. If we have V vertices, for each of them, we test unique pairs of neighbors that can be formed from $d(v)$ items. So, the overall complexity can be expressed

as

$$\begin{aligned}
\text{Complexity} &= \sum_{v \in V} \binom{d(v)}{2} \\
&= \sum_{v \in V} \frac{d(v)!}{2!(d(v)-2)!} \\
&= \sum_{v \in V} \frac{d(v) \cdot (d(v)-1) \cdot (d(v)-2)!}{2 \cdot 1 \cdot (d(v)-2)!} \\
&= \sum_{v \in V} \frac{d(v) \cdot (d(v)-1)}{2}
\end{aligned}$$

In this project, we resorted to the so called **Forward Algorithm** [2], a variant which aims at improving complexity by iterating over vertices in a specific order, and efficiently queries their neighbors to identify potential triangle closures. We first implemented the **sequential** version of the algorithm, using **C++**, in which we can identify three main data structures used:

- **Graph Representation:** How the graph's vertices and edges are stored, enabling efficient retrieval of a node's neighbors. We used and analyzed two primary representations: **adjacency matrices** (implemented as `std::vector<std::vector<int>>`) and **adjacency lists** (implemented as `std::map<int, std::vector<int>>` to handle potentially sparse or non-contiguous node IDs).
- **Auxiliary Sets ($A[t]$):** Data structures used to store a specific subset of neighbors for each node (implemented as `std::vector<std::set<int>>`, where the outer vector is indexed by the node ID). These sets are then intersected (as defined in set theory) to identify the third vertex of a triangle.
- **Ordering/Ranks:** Structures (a sorted list of vertices and a map for ranks) to maintain and query the chosen ordering of vertices. Ranks are crucial for **pruning** and guarantee that each triangle is counted exactly once and that the search space is significantly reduced. Our implementation orders vertices by their degree (from highest to lowest) to optimize performance.

1.1 Forward Node Algorithm V1

In the file `seq_node_it_v1.cpp` there's the implementation using the adjacency matrix. An entry `adjacencyMatrix[i][j]` is 1 if there is an edge between vertex `i` and vertex `j`, and 0 otherwise. So, with N nodes, the space complexity to store the matrix is $\mathcal{O}(N^2)$. Neighbor retrieval is linear, as it's mandatory to scan the full row of the node inside the matrix. The auxiliary set is defined as `std::vector<std::set<int>>` `A`. When processing an edge (s, t) , the algorithm needs to find common elements between `A[s]` and `A[t]`. As `std::set` stores elements in sorted order, `std::set_intersection` can be efficiently applied, resulting in a complexity of $\mathcal{O}(K_0 + K_1)$, being K_0 and K_1 the sets sizes. For the orderings we use `std::map<int, int>` `ranks` and `std::vector<int>` `orderedList`. Checking rank

condition takes $\mathcal{O}(\log N)$ for map lookups, and we iterate through the whole `orderedList` so it takes $\mathcal{O}(N)$.

1.2 Forward Node Algorithm V2 (better version) and comparison

In the file `seq_node_it_v2.cpp` we used the adjacency list, coded as a `std::map<int, std::vector<int>>` to better handle potentially sparse or non-contiguous node IDs. This version is **more efficient**: to find the neighbors of a vertex `u`, the algorithm performs a lookup in the map ($\mathcal{O}(\log N)$) and then copies the associated vector of neighbors ($\mathcal{O}(d(u))$). This is significantly more efficient for sparse graphs than iterating an entire row of an adjacency matrix. So, to resume the comparison between the two algorithms:

Table 1. Summary of Sequential Forward Algorithm Implementation Complexities

Graph Representation	Overall Time Complexity
Adjacency Matrix	$\mathcal{O}(N^3 \log N)$
Adjacency List	$\mathcal{O}(M \cdot d_{max})$

Where N is the number of vertices, M is the number of edges,

and d_{max} is the maximum degree in the graph.

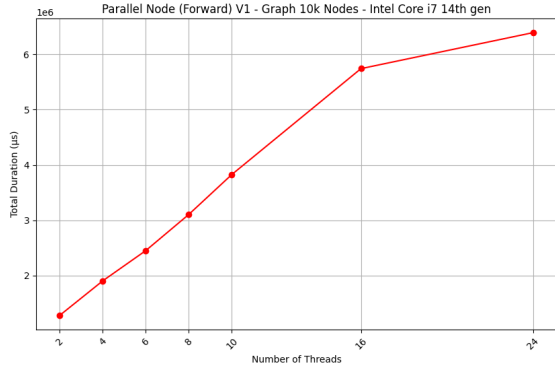
2 Parallel Forward Algorithm

In order to achieve better performances, we created two concurrent versions of the Forward Algorithm which iterates through graph nodes. As we did with the sequential version, the first one uses the adjacency matrix to represent the graph, whereas the latter employs the adjacency list.

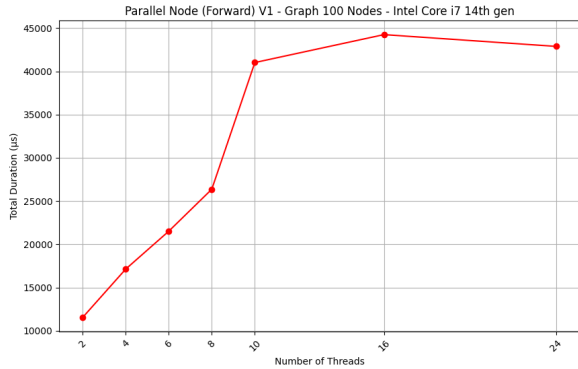
2.1 Parallel Forward Algorithm V1 and related flaws

In the first version of the algorithm, more naive and simple case, the part that gets parallelized is the Forward algorithm function. More in detail, the outer loop that iterates through the nodes in the ordered list is still sequential, running N times, but the **iteration over each node neighbors t is parallelized**: this is achieved by dividing the neighbors of `s` into chunks, with each chunk processed by a separate worker thread using `std::async`. Each worker thread is responsible for computing intersections and updating the auxiliary sets (`A[t]`) for its assigned subset of neighbors. To protect data races and guarantee mutual exclusion over the variable to count the number of triangles and the auxiliary sets, we resorted to **mutexes** (efficiently used with C++ **RAII lock templates**). However, this first version does not improve the execution time of the algorithm compared to the sequential version, and the asymptotic worst-case complexity is unlikely to improve substantially. The mutexes *de-facto* serialize operations on shared data, meaning that even with P threads, the core set operations effectively run in a serialized

fashion due to **contention**. Moreover, the additional overhead related to thread creation (which becomes higher and higher as the number of threads used increases) actually **slows down** the process even more. During our tests, we observed that the excessive overhead was harmful to the performances of the algorithm:



If we lower the number of vertices in the graph to, let's say, just 100, we can see that the parallel version of the algorithm becomes very inefficient, even counterproductive. This is because the time spent creating threads and managing context switching is presumably much higher than the computational gain achieved through parallel execution for such a small problem size. As a result, the overhead negates any potential speedup, making the sequential version the more performant choice:



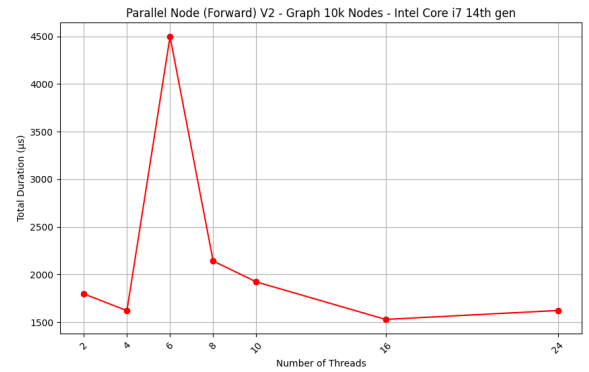
2.2 Parallel Forward Algorithm V2: improvements and scalability limitations

We can now show a much better and efficient version of the Parallel Forward Algorithm. Indeed, this second parallel version achieves **true parallel speedup of the parallelizable portion of the algorithm**, which is the $\mathcal{O}(M \cdot d_{max})$ part. This approach uses an adjacency list graph representation (`std::map<int, std::vector<int>`) (which improves neighbor retrieval time from $\mathcal{O}(N)$ to $\mathcal{O}(\log N + d(v))$) combined with a different parallelization strategy: instead of parallelizing the neighbors scanning for each single node inside each iteration of the algorithm, we now parallelize the outer loop of the Forward Algorithm. The ordered list of vertices is divided into contiguous chunks, and each thread is assigned a chunk of vertices to process independently. So, in other words, each thread

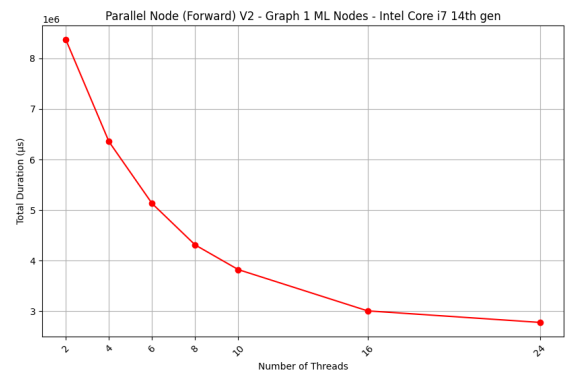
performs the **full triangle enumeration logic for its assigned vertices in a totally independent way**, in parallel with other threads. Also, the total count of triangles (variable `countTriangles`) is managed using `std::atomic<int>`, this significantly **reduces contention** compared to using mutexes for every triangle found. Another perk of this new version is that, since threads primarily operate on independent chunks of the ordered list of node, there is **minimal contention** on other shared data structures(`adjacencyVectors` and `ranks`). Speaking more in depth about the theoretical achievable speedup, given that the $\mathcal{O}(M \cdot d_{max})$ work (from the sequential version) is distributed among **P threads** with very low synchronization overhead (primarily from using atomic variables instead of mutexes), the ideal parallel time complexity can be written as:

$$\mathcal{O}\left(\frac{M \cdot d_{max}}{P} + (M + N \log N)\right)$$

where $(M + N \log N)$ is related to the sequential pre-processing overhead. So, for sufficiently large $M \cdot d_{max}$, **we are able to achieve a significant speedup, proportional to P** (number of thread used), compared to the sequential version. This can be seen also in our experimental tests:



If we compare the two versions of the parallel algorithm, using this same graph, we can see that with every number of threads used we have a significant reduction of time, **approximately 3 orders of magnitude lower**. In addition, using the adjacency list representation gives us the possibility to test much bigger graphs. Here are the results using graphs having 1 million of nodes:

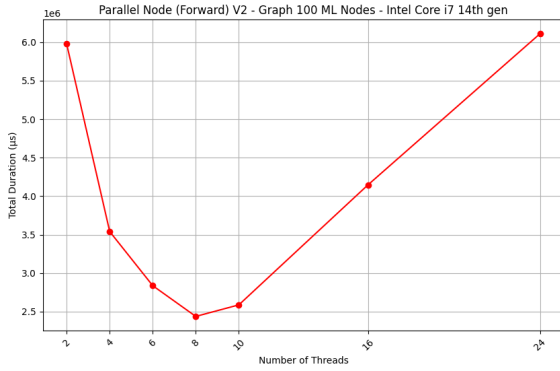


However, if we use even **larger graphs**, we start to see

the potential flaw of this second version: **it doesn't yet reach full scalability**. As a matter of facts, this algorithm does not scale well because of two main defects:

- Each thread performs the same, time-consuming operation of building the ranks map. This is **not a parallelizable part of the code** and severely limits the total speedup, as per **Amdahl's Law**.
- While an atomic variable is better than a mutex, constantly writing to the same memory location from multiple cores causes **contention** and introduces a significant amount of cache coherency overhead, which can degrade performance as the number of threads increases.

For example, with a graph of 100 million nodes, the algorithm achieves its maximum performance with 8 threads. Beyond this point, the performance begins to **degrade**. At 24 threads, the increased overhead is so severe that the program takes longer to complete than with only 2 threads, highlighting the detrimental effects of the sequential bottlenecks and excessive contention.

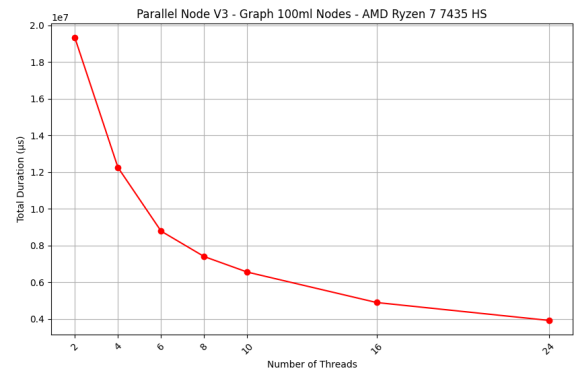
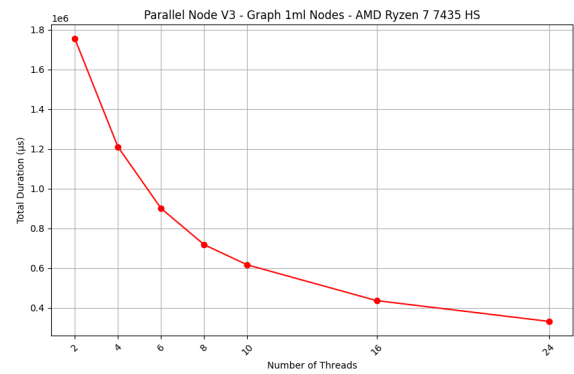
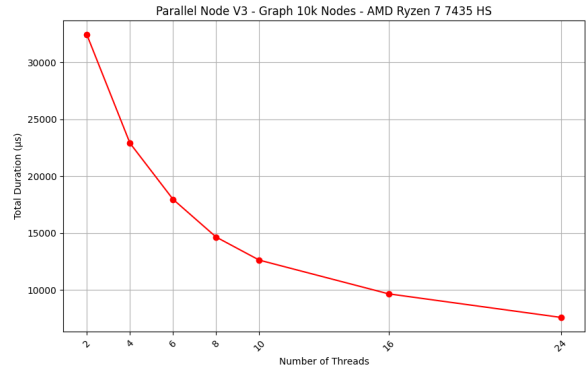


2.3 Parallel Forward Algorithm V3: final version

The final version of our algorithm is the **most efficient and scalable**.

Amdahl's Law states that the speedup of a program from parallelization is limited by the sequential fraction of the program. The new algorithm has a much smaller sequential fraction. The work is mostly done in parallel across the chunks, as the only sequential parts are the loop that launches the workers and the final loop that waits for them to complete. This allows the speedup to continue scaling as you add more threads. In this case, we resorted to **C++ tasks** (by means of `std::async`) instead of threads, to let the OS decide the appropriate time to efficiently schedule them, although we explicitly set the launch policy to `std::launch::async`. The most significant improvement is related to the **ranks creation** which is now executed just once, before launching all the tasks to avoid redundant and time consuming work. Moreover, we introduced a **thread-local count**, summed at the end, which eliminates atomic contention, leading to better scalability with more threads. The core of the algorithm was also re-designed from

the ground up, introducing **unordered sets** to achieve much faster node or rank look-ups when using the adjacency lists or the rank sets (with this solution look-ups now take $\mathcal{O}(1)$ on average, instead of $\mathcal{O}(\log N)$). Using `std::unordered_set` is also beneficial for **neighbor intersections**, which are now performed by means of **two-pointer scans on sorted lists** (in a *merge-like* fashion), achieving a complexity of $\mathcal{O}(k_0 + k_1)$. Compared to the previous approach, this is dramatically faster and more memory-efficient, because it avoids repeated memory allocation. Here are our tests executed using graphs composed of 10k, 1 mln and 100 mln of nodes:

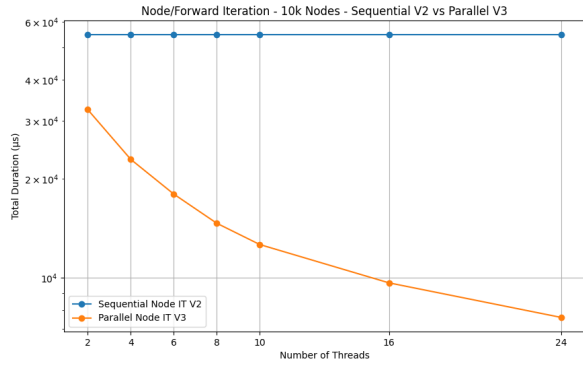


We immediately see the scalability issues are solved: if we increase the thread number, the execution time **always gets lower**. This also represents a substantial performance gain over the V2 version, with a **time reduction of approximately 82-89%** observed across all thread configurations for a 10k node graph:

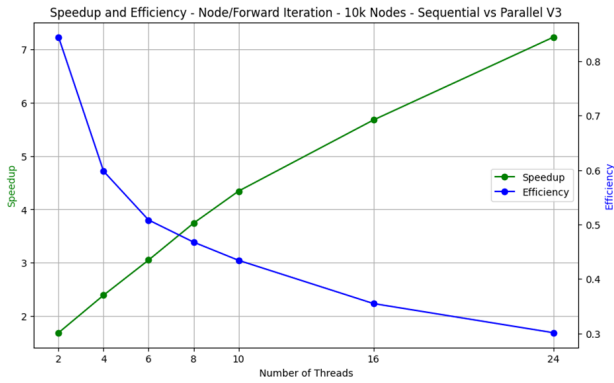
Table 2. Node - Execution times in microseconds and percentage decrease, graph with 10k nodes

#Threads	V2 Time (μ s)	V3 Time (μ s)	Decrease %
2	179 554	32 456	81.92%
4	132 915	22 914	82.76%
6	105 845	17 971	83.02%
8	91 138	14 651	83.92%
10	84 946	12 624	85.14%
16	59 967	9 658	83.89%
24	67 437	7 584	88.75%

Finally, let's compare the best version of the C++ Parallel algorithm with the best version of the sequential algorithm, using a logarithmic scale to analyze the order of magnitude of the **speedup**:



Here is the final chart, displaying the **overall speed-up and efficiency** of the Parallel v2 version versus the sequential version:

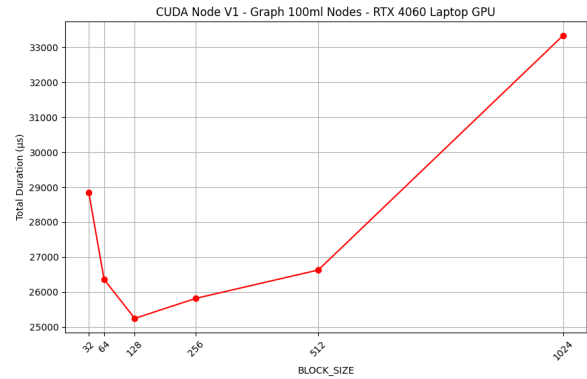


2.4 CUDA Parallel Forward Node Algorithm V1

The inherent massive parallelism of the Forward Algorithm makes it exceptionally well-suited for execution on Graphics Processing Units (GPUs) using NVIDIA's CUDA platform. GPUs, with their thousands of arithmetic logic units (ALUs) and high memory bandwidth, can process a multitude of edges concurrently, offering a significant performance advantage over multi-threaded CPU implementations, especially for very large graphs. To efficiently handle graph data on the GPU, the sparse adjacency list of the graph is converted to **Compressed Row Storage (CRS)** format on the host. This representation consists of two primary arrays: a row pointer array

and a column index array, which are then transferred to the GPU's global memory using `cuda_runtime` library functions. This dense storage format is well-suited for **coalesced** memory access. ON the host side, we initialize the ranking (in a similar way like we did with the V3 version of the C++ Parallel Forward Algorithm). As we already said, this ranking is essential for the parallel algorithm's correctness and efficiency, as it is used to enforce a strict ordering on the traversal of the graph. The core of the algorithm is performed inside the `ForwardAlgorithmKernel` **kernel function**, where we assign **each GPU thread to a graph node**. Within the function, the approach to identify a potential triangle formed by vertices (s, t, v) remains the same as with the C++ versions. Then, To handle concurrent updates to the global triangle counter from multiple threads, the `atomicAdd` function is used. This ensures thread-safe accumulation of the total count. The parallel strategy of the CUDA V1 Forward Algorithm is heavily dependent on the configuration of its launch parameters, particularly the `blockSize`, which serves as a critical **hyperparameter** for performance tuning. As a matter for facts, while the `gridSize` is mathematically derived to ensure all graph nodes are processed, the selection of Block Size involves a trade-off between several factors like warps occupancy or memory utilization.

When we set the `blockSize`, we are essentially telling the GPU how many threads to put into a single block. Since, in current architectures, a warp is composed of **32 threads**, using multiples of 32 can result in the block containing an exact number of full warps, so the GPU can schedule them more efficiently. Here are the result using different configurations, tested on the 100 million nodes graph:



We can see that the best performance is achieved setting `blockSize` to 128, resulting in approximately 25 ms of execution time (this includes also the time spent copying data using from the Host to the Device using the PCIe buffer). Then, the execution time significantly worsens as the size of a block increases to 256, 512, and 1024. In facts, when `blockSize` is small to moderate (i.e. 64 or 128) the blocks are "lightweight" and many can be scheduled simultaneously on an SM. This leads to **high occupancy**, which is crucial for hiding memory access latency. If, on the other hand, we set the size of each block to a much higher number (i.e. 512 or 1024),

the resource requirements for a single block might be so high that the SM can only schedule one or two blocks at a time. This drastically reduces occupancy. Using the NVIDIA interactive kernel profiler, **Nsight Compute**, we can analyze the achieved occupancy, using `blockSize = 128`:

Table 3. Nsight Compute Occupancy Metrics for CUDA Kernel, CUDA Node V1 (`blockSize = 128`)

Metric	Value
Theoretical Occupancy [%]	100.0
Theoretical Active Warps per SM	48
Achieved Occupancy [%]	82.39
Achieved Active Warps per SM	39.55
Block Limit Registers [block]	21
Block Limit Shared Mem [block]	32
Block Limit Warps [block]	12
Block Limit SM [block]	24

So, setting this parameter to 128 results in a block dimension that is sufficiently big to almost **saturate the SM** (achieving an excellent occupancy of 82.39%), but not so large that it requires too many resources per block, which would drastically reduce the number of active blocks at the same time.

2.5 CUDA Parallel Forward Node Algorithm V2

In the second version of the algorithm, we used a **Merge-Like** approach to handle neighbors intersections and look to for the third vertex to complete each triangle. his method uses two pointers (`p1` and `p2`) that traverse the respective neighbor lists, which are stored in the CRS columns `d_adjacencyList_colIdx` array. This approach has a time complexity proportional to the sum of the degrees of `s` and `t`, making it more efficient than a naive nested loop. While this merge-like intersection is a considerable improvement in algorithmic complexity, it introduces potential challenges related to **warp divergence**, a well-known issue in GPU programming. For maximum efficiency, all threads in a warp should execute the same instruction at the same time.

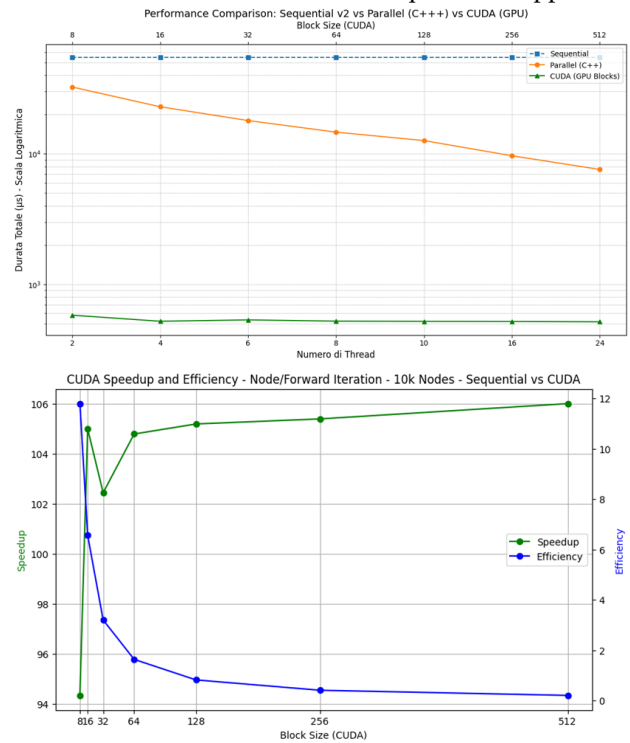
In our kernel, two key areas are susceptible to warp divergence:

1. **Outer Loop Pruning:** The condition `if (rank_s >= d_ranks[t]) continue;` is a point of potential divergence. If some threads in a warp are assigned to nodes `s` that satisfy this condition, while others are assigned to nodes that do not, the warp must execute both the `continue` path and the rest of the loop sequentially. This effectively reduces the parallelism of the warp from 32 threads to a smaller number, or even 1, for that instruction cycle.
2. **Inner Intersection Logic:** The `if-else-if` block within the `while` loop is another critical source of divergence. Threads processing different pairs of neighbor lists (`s, t`) will have varying `v1` and `v2` values. This means that within a single warp, some threads might

take the `v1 == v2` path, while others take `v1 < v2` or `v1 > v2`. The GPU will then serialize the execution of these different paths, executing each branch separately, which diminishes the warp's throughput and performance.

Overall, the second version of the algorithm is able to execute in less time compared to a simple nested loop due to its superior algorithmic complexity and the use of thread-level parallelism. However, the performance is still subject to the overhead of warp divergence caused by conditional branching. This overhead becomes more pronounced in graphs with a wide degree distribution, where the paths and lengths of the neighbor lists vary greatly among different threads within the same warp.

Here are the final charts comparing the **performances** and the **overall speed-up** of the **v2 CUDA** version versus the sequential approach:



3 Forward Edge Iterator Algorithm

The Edge Iterator Algorithm, often referred to as a variant of the Forward Algorithm for triangle counting, takes a different approach to traverse the graph compared to the node-centric iteration. It focuses directly on processing each edge in the graph to identify triangle closures.

A core principle shared with some optimized node iterator algorithms is the use of a specific vertex ordering (ranks), typically based on degree (from highest to lowest), to avoid redundant counting and optimize the search. This ordering creates a "forward" direction for edges and relationships.

In this algorithm, for each edge (u, v) such that $rank(u) < rank(v)$, the algorithm seeks a third vertex w that is a common neighbor of both u and v , and

also satisfies the condition $rank(w) > rank(v)$. If such a w exists, then (u, v, w) forms a triangle. Each triangle is counted exactly once due to this strict rank-based ordering.

To facilitate efficient and non-redundant iteration, the implementation introduces a dedicated `edgeSet`. This set stores all unique edges in the graph. In our C++ implementation, this is achieved using an `unordered_set<Edge>` (where `Edge` is a custom struct with a specialized hash function to handle (u, v) and (v, u) as the same edge). While the `edgeSet` itself doesn't directly enforce the rank condition during its creation, the `EdgeIteratorAlgorithm` function applies the rank check (`if (ranks.at(v0) > ranks.at(v1)) { swap(v0, v1); }` and `if (ranks.at(v) > ranks.at(v1))`) during its iteration to ensure that each triangle is counted exactly once according to the forward algorithm's principles. This allows a clean iteration over graph edges, decoupling the triangle counting logic from the underlying graph representation.

A crucial aspect of this algorithm's efficiency lies in how it finds common neighbors. For each edge (u, v) being considered, the algorithm retrieves the *full* adjacency lists (neighbors) of both u and v . The intersection of these two adjacency lists (`neighborsV0` and `neighborsV1`) yields the set of common neighbors. Each node w in this intersection forms a triangle with u and v , provided the rank condition ($rank(w) > rank(v)$) is met.

Compared to the basic Node Iterator version, the Edge Iterator offers several advantages:

- **Fewer iterations:** Instead of looping over all nodes and then all pairs of their neighbors, the algorithm directly processes each edge. This can significantly reduce the total number of primary iterations, especially in sparse graphs where M (number of edges) is much smaller than N^2 .
- **Direct Edge Processing:** The algorithm's focus on edges makes it naturally suited for sparse graphs, where the work done is proportional to the number of existing connections rather than potential connections.
- **Efficiency with Ordered Neighbors:** When adjacency lists are used and sorted (either explicitly or implicitly through data structures like `std::set` or by sorting `std::vector` neighbors), finding the intersection of neighbor lists can be very efficient, often linear in the size of the smaller list.

As with the Node Iterator, we implemented two versions of the Edge Iterator Algorithm, primarily differing in their underlying graph representation:

- In `seq_edge_it_v1.cpp`, the graph is stored as an adjacency matrix (`std::vector<std::vector<int>>`). Neighbor retrieval involves scanning a row of the ma-

trix, which can be inefficient for sparse graphs. The space complexity is $\mathcal{O}(N^2)$.

- In `seq_edge_it_v2.cpp`, an adjacency list is used, implemented as `std::map<int, std::vector<int>>`. This representation is generally more efficient for sparse graphs. Retrieving neighbors from a `std::map` takes $\mathcal{O}(\log N)$ for the map lookup plus $\mathcal{O}(d(u))$ to access the vector of neighbors. The intersection between neighbor lists (which are `std::vector<int>` here) is performed using `std::set_intersection` after sorting, or by direct two-pointer intersection if the vectors are kept sorted.

Overall, the Edge Iterator Algorithm, especially when combined with an adjacency list representation and degree-based ordering, often provides a more efficient approach to triangle counting in terms of both time and space complexity, particularly for large and sparse graph datasets.

Table 4. Summary of Sequential Edge Iterator Algorithm Implementation Complexities

Graph Representation	Overall Time Complexity
Adjacency Matrix	$\mathcal{O}(N^3)$
Adjacency List	$\mathcal{O}(M \cdot d_{\max})$

Where N is the number of vertices, M is the number of edges,

and $d_{\max}$ is the maximum degree in the graph.

4 Parallel Forward Edge Iterator Algorithm

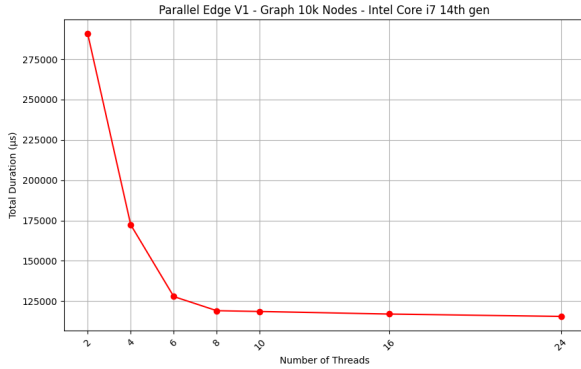
The Parallel Edge Iterator Algorithm is a method to concurrently count triangles in a graph by distributing the workload among multiple threads. This approach is highly effective because the triangle enumeration for each edge is an independent task. We created two primary concurrent versions, each utilizing a different strategy for finding common neighbors between two vertices. **Version 1 (V1)** employs a naive nested-loop intersection, while **Version 2 (V2)** uses a more efficient two-pointer merge-like algorithm. Additionally, we developed a highly optimized **Version 3 (V3)** that incorporates advanced data structures and parallel patterns. Our performance analysis, conducted on an **Intel(R) Core(TM) i7-14650HX (2.20 GHz)** for the first 2 version and on a **Ryzen 7 7435 HS (3.10 GHz)** for the last one, reveals crucial insights into the trade-offs of parallelization and workload size.

4.1 Parallel Forward Edge Iterator V1

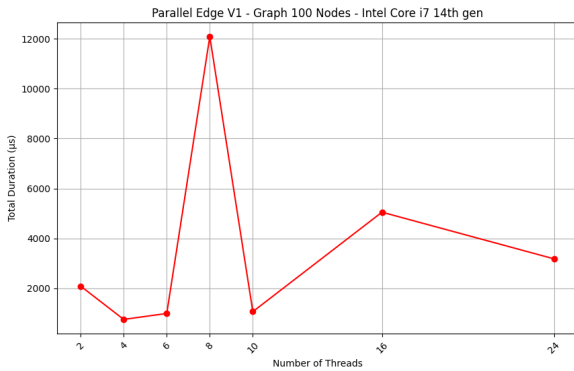
In this more naive version (**Parallel Edge V1**), we parallelize the outer loop that iterates over each edge. We distribute the edges among threads, and each thread is responsible for processing its assigned chunk. For a given edge (s, t) , the thread finds common neighbors by performing a **naive nested-loop intersection** on the neighbor lists of s and t . This method of finding common neighbors has a worst-case time complexity of $\mathcal{O}(d(s) \cdot$

$d(t)$ per edge, where $d(s)$ and $d(t)$ are the degrees of the vertices. The total count of triangles is managed using an `std::atomic<int>`, which significantly **reduces contention** compared to using mutexes.

However, as shown in our experimental tests, the effectiveness of this approach is highly dependent on the graph size. On a larger graph with 10k nodes, the computational workload is substantial enough that the parallel gains outweigh the thread management overhead, leading to a consistent decrease in execution time as the number of threads is increased. The performance flattens out around 8 threads, but importantly, it does not degrade significantly even with more threads, as the workload is not large enough to create substantial overhead.



This is not the case for a smaller graph, such as the one with 100 nodes. On this graph, the computation is so quick that the overhead of creating and managing multiple threads becomes a dominant factor. The graph below shows erratic performance with increasing threads, even degrading significantly at 8 threads, because the operating system is spending more time on context switching and thread scheduling than the threads are spending on actual work. For such small problems, a single-threaded sequential approach is often superior.



4.2 Parallel Forward Edge Iterator V2

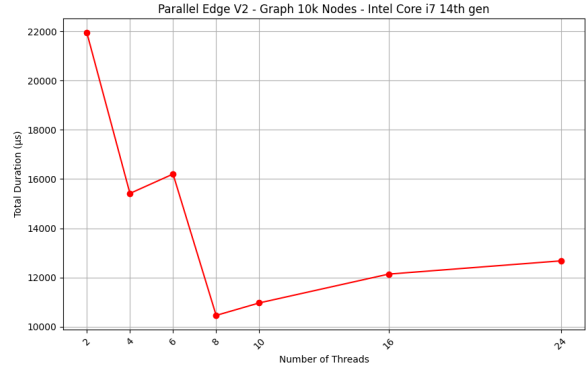
To achieve a more substantial and true speedup, we developed a second, much more efficient version (**Parallel Edge V2**). This version uses an adjacency list representation and employs a crucial preprocessing step to ensure all neighbor lists are sorted. With sorted lists, each thread can find the common neighbors for its as-

signed edges using a **two-pointer merge-like algorithm**, which has a linear time complexity of $\mathcal{O}(d(s) + d(t))$. This significant improvement in the intersection process allows the algorithm to achieve a speedup proportional to the number of threads used, P , with very low synchronization overhead from the `std::atomic` variable. The ideal parallel time complexity can be written as:

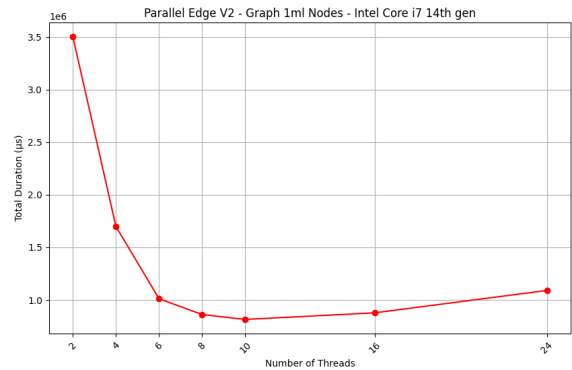
$$\mathcal{O}\left(\frac{\sum_{(s,t) \in E} (d(s) + d(t))}{P} + (M \log N)\right)$$

where $M \log N$ is related to the sequential pre-processing overhead of sorting the neighbor lists. This approach yields significant performance improvements across all graph sizes.

Our analysis of the V2 algorithm on various graph sizes highlights its efficiency and how it scales differently based on workload. On a 10k-node graph, we observe a dramatic and consistent drop in execution time as the number of threads increases, with an optimal point around 8 threads. The performance is consistently better than the V1 version on the same graph, demonstrating that the linear-time intersection is far superior to the naive nested-loop approach.

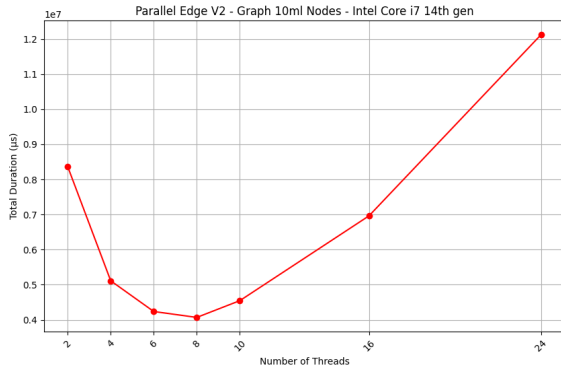


The scaling benefits of the V2 algorithm become even more apparent on extremely large graphs. For a graph with 1 million nodes, the performance curve shows a near-linear decrease as the number of threads increases up to 8. This indicates a near-perfect parallel speedup where the workload is large enough that the benefits of parallelization far outweigh the overhead.



This trend continues with an even larger graph of 10 million nodes, but with a more pronounced effect. The computation time decreases sharply as we increase the

threads up to 8, at which point the algorithm reaches its optimal performance. The intriguing aspect is that for the smaller 10k-node graph, the execution time continues to decrease (or at least flatten out) up to 24 threads, while for the larger 1M and 10M-node graphs, the execution time increases after 8 threads. This is because the massive workload of the larger graphs pushes the system to its limits, and using more than 8 threads (likely the number of high-performance cores) introduces significant overhead from resource contention. When more threads than the optimal number are used, the overhead from context switching and competition for shared resources begins to outweigh the parallelization gains, causing the total duration to increase. This behavior is not seen on the smaller 10k V1 graph because its workload is not large enough to trigger this level of contention.



4.3 Parallel Forward Edge Iterator V3: Final Version

This final version of the Edge Algorithm is similar to the V3 version of the Node/Forward algorithm, with whom it shares significant enhancements. Instead of a standard `std::map`, `std::unordered_map` is used for the adjacency list and ranks. This provides an average-case time complexity of $\mathcal{O}(1)$ for accessing a node's neighbors, which is significantly faster than the $\mathcal{O}(\log N)$ of `std::map`, especially for large graphs. The parallelization is implemented using the `std::async` and `std::future` library, by means of OS-managed tasks. Within each thread, the `threadFunction` function contains its own set of optimizations: we use again **Two-Pointer Intersection** to find common neighbors by resorting to two iterators and advancing them based on which neighbor is smaller. It's important to first sort the neighbors lists of nodes v_0 and v_1 , which is done at the beginning of the program. Overall, this approach is much faster than an $\mathcal{O}(\deg_{v_0} \times \deg_{v_1})$ naive nested loop approach or a hash-set based approach which would have hashing overhead. Also, we get rid of **redundant ranks check** by already pre-filtering the edges so that we only have edges (u, v) where $\text{rank}(u) < \text{rank}(v)$. This means the main loop in thread function does not need to check if $(\text{ranks.at}(v_0) < \text{ranks.at}(v_1))$. By doing this, we *de facto* work with **directed edges**. Moreover, the neighbor lists (`v0_neighbors`, `v1_neighbors`) and

the ranks map are passed by `const` & (**constant reference**), to avoid expensive copying operations. Last but not least, each thread maintains a **local variable to count triangles**. This completely eliminates the need for thread synchronization primitives like mutexes or atomic operations, as there is no shared state being modified concurrently. The results are aggregated at the end, which is a very clean and efficient parallel pattern. Here are the results using 10k, 1 million, and 100 million node graphs:

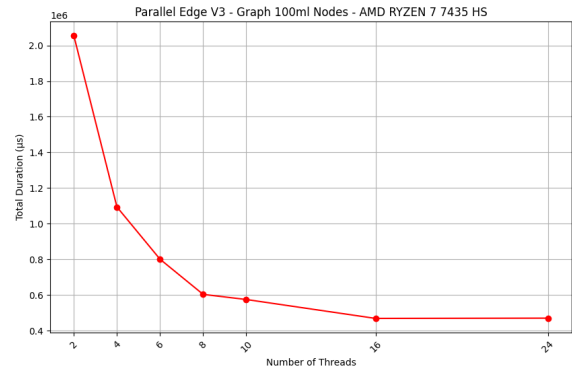
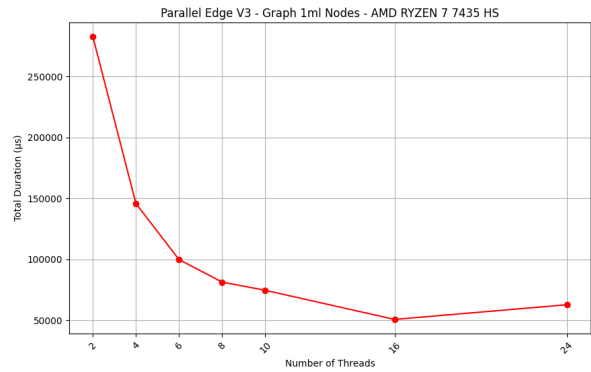
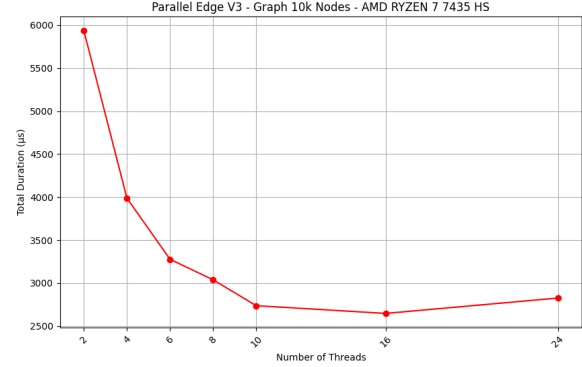
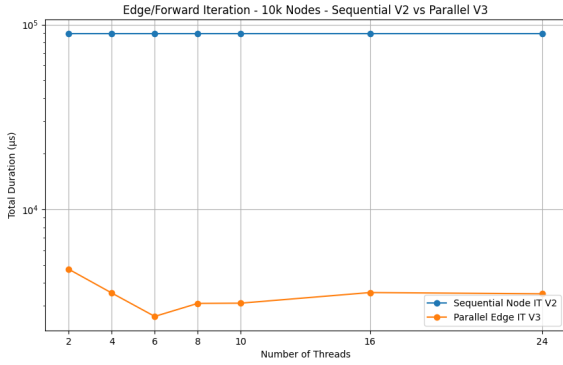


Table 5. Edge - Execution times in microseconds and percent-age decrease, graph with 10k nodes

#Threads	V2 Time (μs)	V3 Time (μs)	Decrease %
2	22012	5972	72.86%
4	15689	3995	74.54%
6	16068	3389	78.90%
8	10258	3052	70.24%
10	11149	2879	74.17%
16	12195	2718	77.71%
24	12734	2937	76.94%

Here is the **final comparison**, using a logarithmic scale, of the performances of the best version of the sequential algorithm versus the best on the parallel algorithm, on a 10k nodes graph:



5 CUDA Parallel Forward Edge Iterator Algorithms

The Edge Iterator method presents a powerful approach to triangle counting on GPUs by assigning a dedicated thread to each edge, (u,v) , in the graph. This strategy effectively distributes the computational workload across the thousands of processing cores available on a GPU. To facilitate efficient data handling and access, the graph's sparse adjacency list is converted to the **Compressed Row Storage (CRS)** format on the host, which is then transferred to the GPU's global memory. The core of the algorithm is implemented within a CUDA kernel function, where each thread independently processes its assigned edge to identify potential common neighbors. To ensure the integrity of the total triangle count, which is updated concurrently by multiple threads, the `atomicAdd` function is employed, effectively preventing race conditions without the performance overhead of mutexes.

Our implementations are structured around two distinct base algorithmic approaches, which are then combined with two key optimization strategies to create a family of increasingly efficient versions. The fundamental difference between our primary algorithmic approaches lies in how each thread finds the common neighbors for its assigned edge (u,v) . **Version 1 (V1)** employs a naive nested-loop intersection, while **Version 2 (V2)** utilizes a more efficient two-pointer merge-like algorithm. On top of these, we apply two optimization strategies: using chunked kernel launches and shared memory.

5.1 V1 Implementations

1. **CUDA Parallel Forward Edge V1:** This is our baseline implementation and the most straightforward version. For a given edge, each thread finds common neighbors by performing a **naive nested-loop intersection** on the adjacency lists of the two connected vertices. The time complexity of this step is $\mathcal{O}(d(u) \cdot$

$d(v))$, where $d(u)$ and $d(v)$ are the degrees of the vertices. This quadratic complexity can be a significant performance bottleneck for edges connected to high-degree nodes. This version launches a single kernel to process all edges in a single, large batch.

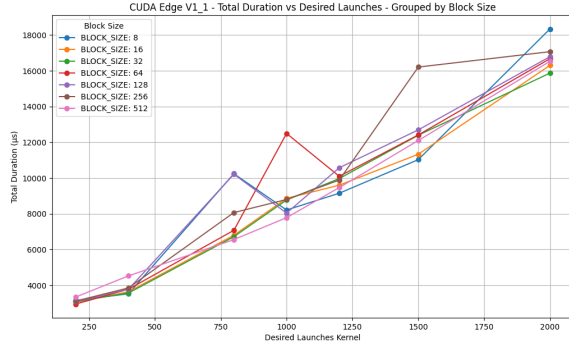
On a massive 100 million-node graph, as shown below, we observe a significant initial decrease in execution time as the `BLOCK_SIZE` increases from 8 to 128. This indicates that increasing the number of threads per block helps to better utilize the Streaming Multiprocessors (SMs) on the RTX 4060 GPU, improving occupancy and hiding memory access latency. However, a sharp increase in execution time is seen at `BLOCK_SIZE` = 256, followed by a slight decrease at `BLOCK_SIZE` = 512. This fluctuating behavior, particularly the spike at 256, suggests that at larger block sizes, the kernel encounters resource limitations (e.g., exceeding register limits or shared memory per block) that reduce the number of active warps or blocks per SM, leading to a significant drop in performance despite the increased thread count. The optimal performance is achieved around `BLOCK_SIZE` = 128 for this specific graph and GPU.



2. **CUDA Parallel Forward Edge V1_1:** This version builds on the V1 algorithm by introducing a **chunked parallelization** strategy. Instead of launching a single kernel to process all edges, the algorithm partitions the edges into smaller chunks and launches a series of kernels, one for each chunk. The goal is to investigate whether this approach can improve GPU resource utilization and latency hiding by allowing the scheduler to manage smaller, more frequent parallel tasks more effectively. The underlying intersection logic within the kernel remains the same naive nested-loop approach as V1.

The graph below illustrates the performance of CUDA Edge V1.1, showing the total duration as a function of the number of kernel launches for various `BLOCK_SIZE` configurations. A clear trend emerges: as the number of kernel launches increases, the total execution time consistently rises across all block sizes. This is due to the inherent overhead associated with each kernel launch, including the time taken for setup, scheduling, and synchronization. While chunking can sometimes improve GPU occupancy, for this algorithm and workload, the overhead of

multiple launches outweighs any potential benefits, especially as the number of launches grows. Interestingly, different `BLOCK_SIZE` values show similar increasing trends, indicating that the overhead of kernel launches is a dominant factor regardless of how many threads are within each block. The lines for `BLOCK_SIZE = 16` and `BLOCK_SIZE = 32` appear to offer slightly better performance at lower launch counts, but the overall penalty for excessive launches is evident across the board.

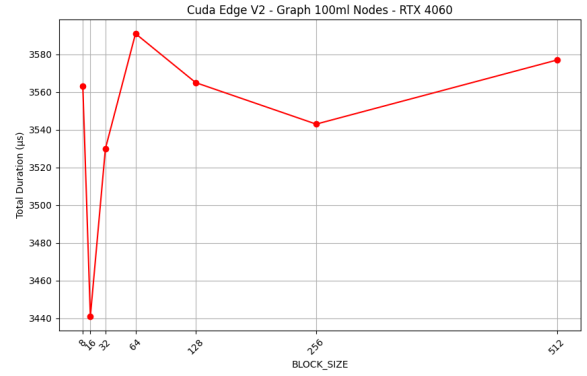


5.2 V2 implementations

1. **CUDA Parallel Forward Edge V2:** This version represents a significant algorithmic improvement over the V1 family. It employs a more efficient **two-pointer merge-like algorithm** for finding common neighbors, which has a linear time complexity of $\mathcal{O}(d(u) + d(v))$. This version assigns one thread to manage one edge and launches a single kernel to process all edges. The efficiency gain comes primarily from the improved intersection algorithm, making it much faster, especially for graphs with high-degree nodes, compared to the V1 implementations.

The performance of CUDA Edge V2 on a 100 million-node graph, as depicted below, shows a distinct characteristic. The execution time initially decreases sharply as the `BLOCK_SIZE` increases from 8 to 32, indicating efficient utilization of GPU resources. However, the curve then shows a slight increase at `BLOCK_SIZE = 64`, followed by a more noticeable increase at `BLOCK_SIZE = 128`, before decreasing again at `BLOCK_SIZE = 256` and finally increasing at `BLOCK_SIZE = 512`. The overall minimum performance is achieved around `BLOCK_SIZE = 32`. This fluctuating behavior suggests that while the merge-like algorithm is efficient, the highest time in V2 (3441 us) is still better than the lowest time in V1 (5534 us), the optimal `BLOCK_SIZE` is critical for maximizing Streaming Multiprocessor (SM) occupancy and minimizing resource contention (e.g., register pressure, shared memory usage) on the RTX 4060. The initial dip indicates the benefit of parallelization, but the subsequent oscillations point to a sensitive balance between

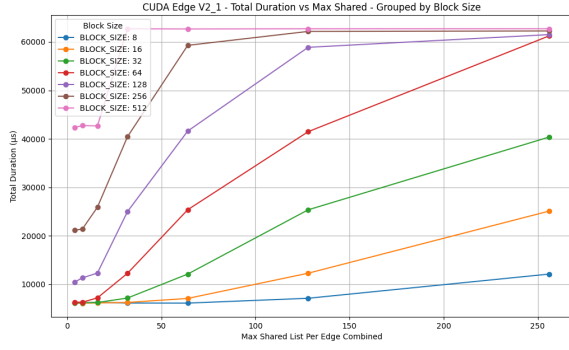
thread count per block and available SM resources.



2. **CUDA Parallel Forward Edge V2_1:** This implementation builds on the V2 algorithm by introducing a specific shared memory optimization. In this version, **each block manages a single edge**. Threads within that block cooperatively load the adjacency lists of the edge's endpoints into shared memory. After a synchronization barrier, a single thread then performs the **two-pointer merge-like intersection** on this faster shared data. This strategy aims to reduce global memory latency for edges whose neighbor lists fit entirely within shared memory, further optimizing the already efficient V2 algorithm.

The graph below illustrates the performance of CUDA Edge V2_1, showing the total duration as a function of "Max Shared List Per Edge Combined" (the maximum combined length of neighbor lists that will be loaded into shared memory) for various `BLOCK_SIZE` configurations. A clear trend is that for most `BLOCK_SIZE` values, increasing the Max Shared List Per Edge Combined generally leads to an increase in total duration. This is because reserving more shared memory per block can reduce the number of active blocks or warps that can run concurrently on an SM, thereby lowering occupancy and overall throughput. Smaller `BLOCK_SIZE` values (e.g., 8 and 16) generally show better performance, especially at lower shared memory thresholds. This suggests that for a single edge processed per block, a smaller block size might be more efficient in terms of resource utilization and occupancy, allowing more blocks to be scheduled simultaneously. The sharp increases for larger `BLOCK_SIZE` values (e.g., 256, 512) indicate that exceeding certain shared memory limits per block can severely degrade performance due to resource constraints. This highlights the critical balance between

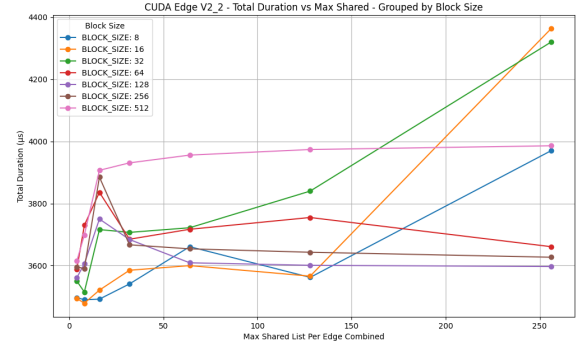
shared memory usage and overall GPU occupancy.



3. **CUDA Parallel Forward Edge V2_2:** This version also utilizes the efficient **two-pointer merge-like algorithm** and incorporates **shared memory**, but with a different thread-to-edge mapping compared to V2_1. Here, **one thread manages one edge**, similar to the base V2. However, each thread attempts to load the adjacency lists for its assigned edge into a dynamically allocated portion of shared memory before performing the intersection. This allows multiple edges to be processed concurrently within a single block, each benefiting from faster shared memory access if their neighbor lists are small enough.

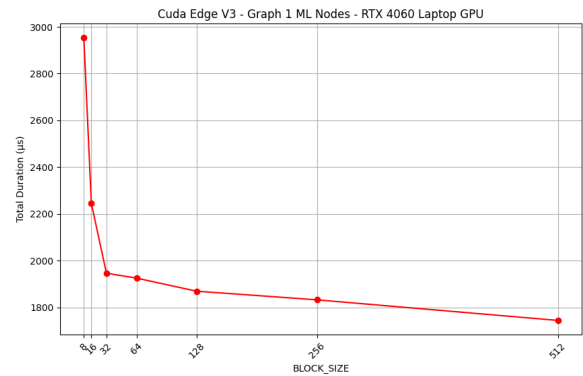
The graph below presents the performance of CUDA Edge V2_2, showing the total duration against "Max Shared List Per Edge Combined" for different `BLOCK_SIZE` values. This graph is notably more complex than V2_1, indicating a more intricate interplay between `BLOCK_SIZE` and shared memory usage. For smaller `BLOCK_SIZE` values (e.g., 8, 16, 32), the performance tends to be lower (faster execution) and less sensitive to changes in Max Shared List Per Edge Combined at lower values. As Max Shared List Per Edge Combined increases, the duration generally rises, especially for larger `BLOCK_SIZE` values. This reinforces the idea that excessive shared memory allocation per thread can limit the number of concurrent blocks, thereby reducing overall GPU utilization. The optimal performance appears to be achieved with smaller `BLOCK_SIZE` values and lower shared memory thresholds, where the benefits of faster memory access are maximized without incurring significant resource contention or limiting concurrency. The varying slopes and intersections of the lines suggest that finding the optimal balance for `BLOCK_SIZE` and Max Shared List Per Edge Combined is crucial for maximizing the efficiency of this shared-

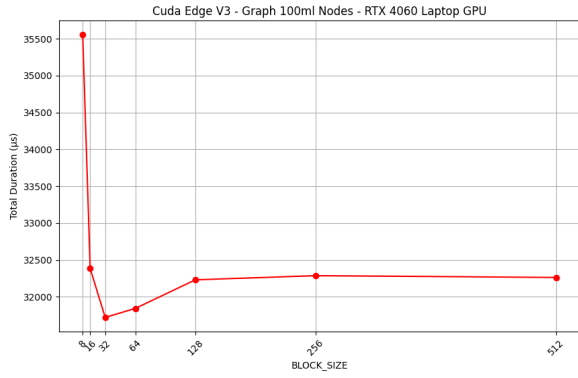
memory optimized, one-thread-per-edge approach.



5.3 V3 implementations

The main problem encountered in the previous versions of the algorithm was the **lack of scalability**: the program was relatively fast, however, if we incremented the number of threads within each block, the performances got worse. We tried to overcome this issue with the V3 version of the program, in which we created a Kernel function implementing a **dynamic work queue for edge iteration**. This kernel processes edges in a dynamic manner, allowing threads to fetch work from a global queue. This way, threads can dynamically take on work as they finish processing previous edges, **improving load balancing** and potentially reducing idle time: a "lucky" thread which had to process an edge of lower degree nodes does not remain inactive once it has finished its work, but can instantly process another edge, and so on. In other words, faster threads can take on more work, while slower threads will naturally catch up, leading to a more efficient use of GPU resources. However, this noticeable improvement in scalability and robustness comes at a cost: **we sacrifice execution time**, which increases approximately by a factor of 10 using the 100 million nodes graph (i.e, going from approx. 3 ms to approx. 30 ms). This slowdown is related to the nature of the global dynamic queue and especially to the way workers access it (i.e. by performing `atomicAdd`, which introduces atomic contention between thousands of threads. Look at table 6 for a more detailed comparison of the versions of the program. Here's the performance observed using a 1 million and a 100 million nodes graphs:





We can observe that, when increasing the block size, the performances either get better or reach a *plateau*, this is a huge improvement over the previous versions, which saw a dramatic decrease in execution speed. To lower the execution time, we can also resort to a hybrid approach which puts together the strengths of all the approaches. In the V3_1 version of the algorithm we introduced a hybrid kernel with static assignation between blocks, and dynamic for the threads inside each block, using **shared memory**. More in details, we use shared memory to store a block-level counter which acts as index for the block work queue, then each thread performs a dynamic work cycle by pulling the work index from the work queue in the shared memory. Using the same graph and setting at 8 the block size parameter, this second approach lets us save almost **15%** of execution time (going from 35 ms to 30 ms).

5.4 CUDA Forward Edge Performance Summary

The performance analysis of the CUDA Edge Iterator algorithms reveals distinct characteristics across its versions. **CUDA Edge V1**, utilizing a naive nested-loop intersection, shows an initial performance improvement with increasing `BLOCK_SIZE` on a 100 million-node graph, reaching an optimum around $5500\mu s$ at `BLOCK_SIZE = 128`, but then degrades due to resource contention at larger block sizes. **CUDA Edge V1_1**, which introduces chunked kernel launches to V1's core, generally exhibits increased total duration with more launches, indicating that kernel launch overhead often outweighs any benefits from finer-grained task management. In contrast, **CUDA Edge V2** and **V2_2** (both employing the efficient two-pointer merge-like algorithm with one thread per edge) achieve significantly superior performance, consistently operating within the $3400\mu s$ to $4400\mu s$ range, underscoring the importance of algorithmic efficiency. **V2_2** further optimizes with shared memory, though its peak performance is sensitive to `BLOCK_SIZE` and shared memory allocation. Conversely, **CUDA Edge V2_1**, which maps one block per edge and uses shared memory, generally performs worse than other V2 variants, with times ranging from $5000\mu s$ and escalating dramatically. This suggests that its block-to-edge mapping and shared memory strategy can lead to poor GPU

occupancy and inefficient resource utilization, negating its algorithmic advantages.

6 Matrix Multiplication Algorithms

For the Matrix Multiplication algorithm, we choose to immediately start talking about the C++ parallel version, without mentioning sequential approaches. This is because the sequential version of the matrix cubing steps of the algorithm would have a very high time complexity, making it inefficient for large graphs. Since we are required to multiply matrix A by itself for three times, in the worst case the complexity would be $\mathcal{O}(n^3)$. So, in short words, a parallel version of the algorithm is much needed in this case.

6.1 Parallel Matrix Multiplication

Although the overall computational workload (the total number of operations) remains the same, with the parallel version we aim to improve the **practical performance** (i.e. the execution time) by using distributing all the work across multiple threads. More precisely, each matrix multiplication is divided among p threads. The work is split by rows, with each thread handling a chunk of rows. The time it takes for a single thread to complete its portion of the work is roughly the total work divided by the number of threads, so $\mathcal{O}(n^3/p)$. Then, since all threads are working concurrently, the total execution time is determined by the longest-running thread. In a balanced workload, this is the time it takes for any single thread to finish its work. Therefore, the time complexity of the parallelized algorithm is, assuming a balanced workload, approximately:

$$\mathcal{O}(n^3/p)$$

The C++ code implements this parallel strategy to compute A^3 in two distinct stages:

1. Stage 1: Computing $A^2 = A \times A$

The algorithm launches p threads, each responsible for computing a specific range of rows of the resulting A^2 matrix. A synchronization barrier (via `join()`) ensures that all threads complete their work before moving to the next stage.

2. Stage 2: Computing $A^3 = A^2 \times A$

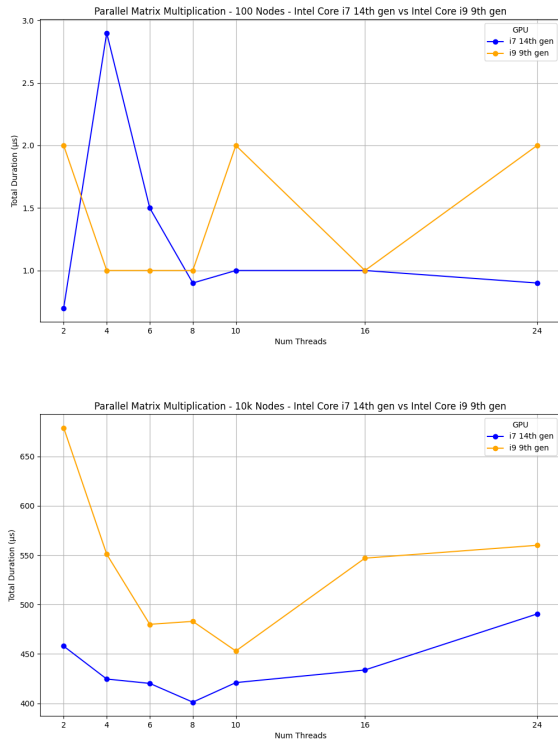
After all the threads waited on the **barrier**, they can go ahead and use the previously computed A^2 matrix and the original A matrix as input. A final synchronization barrier ensures the completion of this second multiplication before the final A^3 matrix is returned.

A key optimization for sparse matrices is included within the core multiplication function. By checking for zero elements (`if (A[i][k] != 0)`), the algorithm **avoids unnecessary computations**, leading to significant performance gains on average-case scenarios for sparse graphs. At the end, we linearly compute the **Trace** of the resulting matrix and multiply it by $\frac{1}{6}$. Because we used the **adjacency matrix** to represent

Characteristic	Version V1/V2	V3 (Global Dynamic)
Task Assignment	Local mathematical calculation	Atomic access to a global queue
Assignment Overhead	Almost Zero	Significant (contention on <code>atomicAdd</code>)
Load Balancing	Poor (depends on the data)	Excellent
Scalability with <code>BLOCK_SIZE</code>	Poor (performance worsens)	Good (performance scales)
Absolute Performance	Higher (in its sweet spot)	Lower (due to overhead)

Table 6. Comparison of various versions of CUDA Edge Algorithm.

the graphs (and we’re forced to do so if we want to apply this algorithm), due to RAM limitations we cannot test too large graphs, so we just considered the 100 node and 10k node graphs. Here are the results:



We can say that the 100 Nodes graphs does not offer many insights as the graph size is too small, and threads overhead time has a lot of influence on the performances. Speaking about the 10k Nodes graph, we can see that for both processors (Intel Core i7 14th gen, Intel Core i9 9th gen) we have a minimum at around 8-10 threads. As for the 10k Nodes graph, the Parallel Matrix Multiplication Algorithm is **the fastest among all the tested algorithms**, even compared to the CUDA ones.

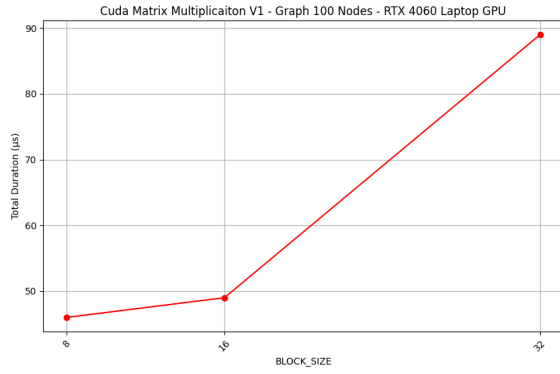
7 Cuda Matrix Multiplication

As an alternative approach to triangle counting, we have developed two distinct CUDA-accelerated matrix multiplication algorithms, building upon the fundamental principle that the number of triangles in a graph can be derived from the trace of its adjacency matrix cubed, specifically $\text{trace}(A^3)/6$. These implementations, designated as Version 1 (V1) and Version 2 (V2), primarily differ in their optimization strategies for the core matrix multiplication operation. V1 employs a straightfor-

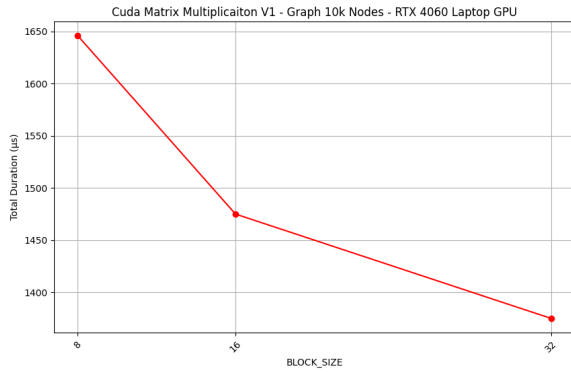
ward, global-memory-centric approach controlled by a single `BLOCK_SIZE` parameter, whereas V2 introduces a more advanced tiled multiplication technique leveraging shared memory, parameterized by `TILE_SIZE`, with the trace calculation’s block size separately configurable via `TRACE_BLOCKSIZE`. Due to concerns regarding GPU memory limitations, we only tested these algorithms on graphs with 100 and 10k nodes.

The **CUDA Matrix Multiplication V1** algorithm implements a direct row-major matrix multiplication. In its `multiplyMatricesKernel`, each GPU thread is assigned the responsibility of computing a single element of the resulting matrix C . This is achieved by mapping threads to specific (row, col) indices within the output matrix, calculated from their `blockIdx` and `threadIdx`. Each thread then iterates through the k dimension, performing the dot product of the corresponding row from matrix A and column from matrix B . While conceptually simple, this approach inherently involves frequent accesses to global memory. Specifically, accesses to matrix B are often uncoalesced, leading to suboptimal memory bandwidth utilization. The overall matrix multiplication is performed in two steps: first $A \times A$ to get A^2 , and then $A^2 \times A$ to get A^3 . The trace of A^3 is subsequently computed by a separate `traceKernel`, where individual threads sum diagonal elements using `atomicAdd` to ensure thread-safe accumulation into a global counter.

The performance of CUDA Matrix Multiplication V1 varies significantly with graph size. For a **100-node graph**, as shown below, the total duration increases as `BLOCK_SIZE` increases. This counter-intuitive behavior for a small graph suggests that the overhead associated with launching larger blocks and managing more threads per block, combined with potential underutilization of the GPU for such a small workload, outweighs any benefits of increased parallelism. The computation is so minimal that the overhead becomes dominant.

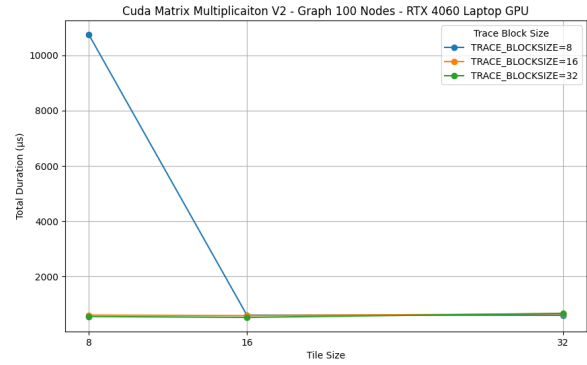


Conversely, for a **10k-node graph**, the total duration consistently decreases as `BLOCK_SIZE` increases from 8 to 32, as depicted in the graph below. This indicates that for a larger and more substantial workload, increasing the `BLOCK_SIZE` effectively utilizes the GPU's Streaming Multiprocessors (SMs), improving occupancy and hiding memory access latency, which are crucial for performance in compute-bound tasks.

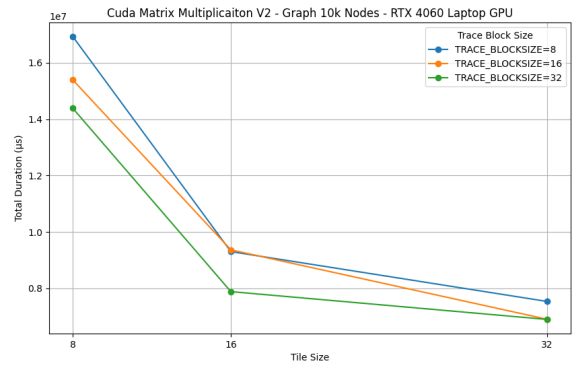


In contrast, **CUDA Matrix Multiplication V2** significantly enhances performance by implementing a **tilled matrix multiplication** strategy that extensively utilizes **shared memory**. The `multiplyMatricesSharedMemoryKernel` partitions the matrices into smaller, square tiles, with each thread block responsible for computing a sub-matrix of the output. Within a block, threads cooperatively load the necessary tiles of matrices *A* and *B* from global memory into fast, on-chip shared memory (`s_A` and `s_B`). This process is synchronized using `__syncthreads()` to ensure all data is available before computation proceeds. The dot product for each output element is then performed using the data residing in shared memory, drastically reducing global memory traffic and improving data locality. The `TILE_SIZE` parameter dictates the dimensions of these shared memory tiles and, consequently, the block dimensions. This tiling approach is iterated over the entire matrix, processing one tile at a time. Similar to V1, the trace of the final A^3 matrix is computed by a dedicated `traceKernel`, but in V2, the `TRACE_BLOCKSIZE` for this operation is also a configurable parameter, allowing for fine-tuning of the trace calculation's parallel execution.

The performance of CUDA Matrix Multiplication V2 demonstrates the substantial benefits of shared memory tiling. For the **100-node graph**, as shown below, the total duration drops sharply as `TILE_SIZE` increases from 8 to 16, and then remains consistently low. This indicates that even for small graphs, the efficiency gained from reducing global memory accesses and improving data locality through tiling significantly outweighs any overhead, leading to very fast execution times. The impact of `TRACE_BLOCKSIZE` is minimal here, as the trace computation is a small portion of the overall workload for such a small matrix.



For the **10k-node graph**, the performance improvements are even more pronounced, as depicted in the graph below. The total duration consistently decreases as `TILE_SIZE` increases, with the lowest times achieved at `TILE_SIZE = 32`. This confirms that for larger, more computationally intensive tasks, the tiled approach with shared memory is highly effective in maximizing GPU throughput. The different lines for `TRACE_BLOCKSIZE` show that while `TILE_SIZE` has the primary impact, optimizing the trace calculation's block size can further refine the overall performance, with `TRACE_BLOCKSIZE=32` generally yielding the best results.



8 REFERENCES

- [1] Schank, T., & Wagner, D. (2005). Finding, Counting, and Listing all Triangles in Large Graphs, An Experimental Study
- [2] In WEA 2005: Experimental and Efficient Algorithms (pp. 606-609). Springer

[3] Cohen, J. (2009). Graph Twiddling in a MapReduce World. *Computing in Science & Engineering*, 11(4), 29-41

THE AUTHORS



Michele Carloni



Riccardo Revalor

insert your text here

test

Computer Engineering MSc student, Politecnico di Torino. Computer Science MSc student, University of Illinois at Chicago.

LinkedIn: [linkedin.com/in/riccardo-revalor/](https://www.linkedin.com/in/riccardo-revalor/)

GitHub: github.com/RiccardoRevalor
